

N100 Financial Intelligence Platform

Analyst Guide

Covers: the Streamlit screener, dashboard navigation, PDF report generation, the REST API, and troubleshooting. Written for an analyst using the platform day-to-day -- not a developer setup guide.

Sprint 6 — Bluestock Fintech — Darshan Kumar

Contents

1. Getting Started	3
2. Using the Streamlit Screener	4
3. Dashboard Navigation	5
4. Generating PDF Reports	6
5. API Usage	7
6. Troubleshooting	9
7. Known, Documented Data Limitations	10

This guide documents the platform as built through Sprint 6 (API server, KMeans clustering, and final QA). It reflects real, verified behavior confirmed against the running system — not the original specification's wording where the two differ. Where that happens, the difference is called out explicitly rather than hidden.

1. Getting Started

The platform has three parts you can run independently: the Streamlit dashboard (interactive screens for browsing companies), the FastAPI server (a REST API for the same underlying data), and a set of report-generation scripts (PDF tearsheets, sector reports, a portfolio summary). All three read from the same SQLite database at `data/nifty100.db`.

Starting the dashboard

```
source .venv/Scripts/activate
streamlit run src/dashboard/app.py
```

Opens at `localhost:8501`. Each screen in the sidebar is its own page under a stable URL, e.g. `localhost:8501/profile`, `localhost:8501/screener`.

Starting the API

```
uvicorn src.api.main:app --port 8000
```

Opens at `localhost:8000`. Interactive API documentation (Swagger UI) is auto-generated at `localhost:8000/docs` — this is the fastest way to try an endpoint without writing curl commands by hand.

Both can run at the same time, on their own ports, with no conflict — confirmed directly during Sprint 6 performance testing.

Rebuilding the database

```
python db/loader.py
```

Safe to rerun any time — drops and recreates `nifty100.db` from the raw Excel source files. Needed only after source data changes; not part of normal day-to-day use.

2. Using the Streamlit Screener

The Screener screen (`localhost:8501/screener`) filters all 92 companies against up to 10 financial metrics at once, using the same outlier-guarded, tested filtering logic as the API's `/api/v1/screener` endpoint.

Quick Presets

Six one-click presets pre-fill sensible filter combinations:

- **Quality Compounder** — ROE $\geq$ 15%, D/E $\leq$ 1.0, positive FCF, Revenue CAGR $\geq$ 10%
- **Value Pick** — P/E $\leq$ 20, P/B $\leq$ 3.0, D/E $\leq$ 2.0, Dividend Yield $\geq$ 1%
- **Growth Accelerator** — PAT CAGR $\geq$ 20%, Revenue CAGR $\geq$ 15%, D/E $\leq$ 2.0
- **Dividend Champion** — Dividend Yield $\geq$ 2%, positive FCF
- **Debt-Free Blue Chip** — D/E effectively 0, ROE $\geq$ 12%
- **Turnaround Watch** — Revenue CAGR $\geq$ 10%, positive FCF

Clicking a preset fills in only the sliders it names — every other filter stays off. Click **Reset all filters** in the sidebar to clear everything and start over.

Manual filtering

10 sliders in the left sidebar cover ROE, D/E, FCF, Revenue CAGR (5yr), PAT CAGR (5yr), OPM, P/E, P/B, Dividend Yield, and ICR. A slider only applies once you move it off its default (off) position — leaving a slider untouched means that metric isn't filtered at all.

Reading results and exporting

The results table is sorted by `composite_quality_score`, highest first. Use the **Download results as CSV** button beneath the table to export the currently filtered list.

On the outlier guard

Three companies — HAL, BEL, and INDIGO — have mathematically correct but extreme ROE/ROCE figures because of an unusually small equity base relative to profit (HAL's raw ROE is over 3,800%). The screener automatically excludes these three from any ROE/ROCE-based ranking or filter so they don't distort results — this is intentional, tested behavior, not missing data.

Interpreting the results table columns

- **composite_quality_score** — a single sector-relative score blending multiple ratios; used purely to sort results, not a filterable metric.
- **return_on_equity_pct / debt_to_equity** — latest fiscal year values, never a TTM (trailing twelve month) figure — TTM rows are excluded everywhere in the platform because they represent a rolling window, not a fixed year, and would otherwise sort incorrectly against real fiscal years.
- **free_cash_flow_cr** — in crores, latest fiscal year.
- **revenue_cagr_5yr / pat_cagr_5yr** — 5-year compound annual growth rates; null for companies with fewer than 5 years of usable history (e.g. JIOFIN).
- **pe_ratio / pb_ratio / dividend_yield_pct** — sourced from market cap / valuation data, not the ratio engine's own computed fields.

3. Dashboard Navigation

The dashboard has 8 screens, listed in the left sidebar in this order:

Home — /

Landing page and platform overview.

Profile — /profile

Search any company by name or ticker. Shows sector, sub-sector, description, 6 headline KPI tiles (ROE, ROCE, Net Profit Margin, D/E, Revenue CAGR 5yr, FCF), and multi-year Revenue/Net Profit and ratio charts.

Screener — /screener

Filter all 92 companies against up to 10 metrics, with 6 quick presets. See Section 2.

Peers — /peers

Compare a company against its peer group across percentile-ranked metrics (11 real peer groups, tested in Sprint 3).

Trends — /trends

Multi-year trend charts for a selected company's core ratios.

Sectors — /sectors

Sector-level breakdowns across the 10 real broad sectors in the data (see the note below on sector count).

Capital — /capital

Capital allocation patterns — which companies are Reinvestors, Distributors, showing Distress Signals, and so on, per Sprint 2/5's 8-pattern classifier.

Reports — /reports

Entry point for downloading generated PDF reports (tearsheets, sector reports, portfolio summary) once they exist on disk. See Section 4 for how those files are generated.

A note on sector count: the project specification refers to 11 sectors in a few places. The real, repeatedly-verified number in the data is 10 — there is no 11th sector with any company assigned to it. The Sectors screen, the API's `/api/v1/sectors` endpoint, and the test suite all correctly show 10. This is documented, verified behavior, not a bug.

4. Generating PDF Reports

Three kinds of PDF reports exist: per-company tearsheets, per-sector reports, and a single portfolio summary. All are generated from the command line, not from inside the dashboard — the dashboard's Reports screen only lets you download files that already exist.

Company tearsheets

A tearsheet is a 2-page PDF for one company — KPI tiles, Revenue/Net Profit and ROE/ROCE charts, balance sheet composition, a cash flow waterfall, auto-generated pros/cons, and a capital allocation badge.

Generate a single company's tearsheet:

```
python src/reports/tearsheet.py --ticker INFY
```

Saves to `reports/tearsheets/INFY_tearsheet.pdf`. Works for any of the 92 companies with at least 3 years of ratio history — if a ticker doesn't qualify, the script prints why instead of producing a broken file.

Regenerate all 92 tearsheets:

```
python src/reports/generate_tearsheets_batch.py
```

Companies with fewer than 3 years of data are skipped and logged to `output/skipped_tearsheets.csv`. Currently this means **91 of 92** tearsheets get generated — JIOFIN is skipped because it only has 2 years of history in the source data. This is expected, documented behavior, not a bug.

Running `python src/reports/tearsheet.py` with no arguments regenerates a fixed 5-company test set (TCS, HDFCBANK, RELIANCE, SUNPHARMA, TATASTEEL) — useful for quickly checking layout changes across different sectors without a full 92-company batch run.

Sector reports

```
python src/reports/sector_report.py
```

Generates all 10 sector PDFs to `reports/sector/` in one run — each with a sector summary page (median KPIs) and a table of every company in that sector.

Portfolio summary

```
python src/reports/portfolio_summary.py
```

Generates one PDF, `reports/portfolio/portfolio_summary.pdf`, with one page per company (all 92, alphabetical by ticker), each showing top 6 KPIs and a trend indicator (UP/DOWN/FLAT, shown as plain text labels rather than arrow symbols, since ReportLab's default font can't render Unicode arrow glyphs).

None of the four report scripts currently take a full range of CLI options — `sector_report.py` and `portfolio_summary.py` always regenerate everything in one run; there's no single-sector or single-page mode yet. If you need that, it would be a similar small addition to what was done for `tearsheet.py`'s `--ticker` flag.

5. API Usage

The API exposes the same underlying data as the dashboard, under `/api/v1`, for programmatic use. Start it with `uvicorn src.api.main:app --port 8000` and try any endpoint interactively at `localhost:8000/docs` before scripting against it.

Health check

```
curl -s http://127.0.0.1:8000/api/v1/health
```

Returns status, row counts for every table in the database, uptime, and a version string. Good first call to confirm the server and database are both reachable.

Company data

```
curl -s http://127.0.0.1:8000/api/v1/companies/TCS
curl -s "http://127.0.0.1:8000/api/v1/companies/TCS/ratios?year=2024-03"
curl -s http://127.0.0.1:8000/api/v1/companies/TCS/pl
curl -s -o TCS_tearsheet.pdf
http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet
```

The last example downloads the pre-generated tearsheet PDF directly — the tearsheet must already exist on disk (see Section 4); the API serves the file, it doesn't generate it on the fly.

Screener

```
curl -s "http://127.0.0.1:8000/api/v1/screener?min_roe=15"
curl -s "http://127.0.0.1:8000/api/v1/screener?sector=Information+Technology&max_de=1.0"
```

Response shape is `{"count": int, "filters_applied": {...}, "results": [...]}` — not a bare array. If you're parsing results programmatically, read `response["results"]`, and use `response["count"]` for the total match count rather than measuring the response length yourself.

Sector names must match the real stored values exactly — e.g. `Information Technology`, not `IT`. An unrecognized sector value returns HTTP 400.

Peers, sectors, and portfolio stats

```
curl -s http://127.0.0.1:8000/api/v1/peers/IT_Services
curl -s http://127.0.0.1:8000/api/v1/sectors
curl -s http://127.0.0.1:8000/api/v1/portfolio/stats
```

`/portfolio/stats` returns pre-computed P10 through P90 percentiles for 10 core KPIs across all 92 companies — it reads directly from `output/portfolio_stats.csv` rather than recomputing on each request, so it stays fast regardless of load.

Document links

```
curl -s http://127.0.0.1:8000/api/v1/companies/TCS/documents
```

Returns each annual report link along with an `is_url_valid` flag. This endpoint checks links concurrently and takes roughly 9 seconds for a company with many documents — noticeably slower than the other endpoints, which is expected given it makes a live outbound request per link rather than reading only from the local database.

All endpoints

Method	Endpoint	Description
GET	/api/v1/health	Server status, DB row counts, uptime
GET	/api/v1/companies	List all companies; filter by sector, market cap category, search
GET	/api/v1/companies/{ticker}	Full company profile + latest KPIs
GET	/api/v1/companies/{ticker}/pl	P&L history (from_year/to_year params)
GET	/api/v1/companies/{ticker}/bs	Balance sheet history
GET	/api/v1/companies/{ticker}/cashflow	Cash flow history
GET	/api/v1/companies/{ticker}/ratios	Computed KPIs per year (optional year param)
GET	/api/v1/companies/{ticker}/tearsheet	Pre-generated tearsheet PDF (binary download)
GET	/api/v1/screener	Ranked, filtered company list
GET	/api/v1/sectors	All 10 sectors with median metrics
GET	/api/v1/sectors/{sector}/companies	Companies in a sector
GET	/api/v1/peers/{group_name}	Peer group with percentile ranks
GET	/api/v1/companies/{ticker}/peers/compare	Radar data vs. peer group + benchmark
GET	/api/v1/market-cap/{ticker}	Historical P/E, P/B, EV/EBITDA, div. yield (2019-2024)
GET	/api/v1/portfolio/stats	P10-P90 percentile table across 10 core KPIs
GET	/api/v1/companies/{ticker}/documents	Annual report links with URL-validity flag

6. Troubleshooting

The API returns 400 for a sector I'm sure exists

Check the exact spelling and spacing against the real 10 values: Communication Services, Consumer Discretionary, Consumer Staples, Energy, Financials, Healthcare, Industrials, Information Technology, Materials, Real Estate. Short forms like "IT" or "FMCG" are not recognized.

A screener result looks wrong for a specific company (e.g. an extreme ROE)

Check whether the company is HAL, BEL, or INDIGO first — these three have a known, intentional outlier guard applied to ROE/ROCE-based filters and rankings because of a small-equity-base distortion, not a data error.

A company's balance-sheet-dependent metrics (ROE, ROCE, D/E) show as N/A

If the ticker is SBIN, this is expected — SBIN has no rows in the balancesheet table, confirmed isolated to SBIN alone among all 92 companies. Every other company should have full data; if another ticker shows this, that's worth investigating as a real issue.

A chart or table looks empty for a recently-listed or thin-history company

JIOFIN has only 2 years of history in the source data, which means 5-year CAGR fields are correctly null and it's skipped from batch tearsheet generation. This is a data-coverage limitation, not a platform bug.

Streamlit or the API won't start / port already in use

Only one process can bind a given port. If port 8501 or 8000 is already in use (e.g. a previous run didn't shut down cleanly), stop the earlier process first, or start the new one on a different port with `--port <number>`.

Shell commands don't work as expected on Windows

Check which terminal you're using. Git Bash (prompt ending in `$`) accepts Unix-style commands like `ls -la`. PowerShell (prompt `PS C:\...>`) does not — use `Get-ChildItem` or plain `ls` there instead.

A pip install fails or a package seems missing

Confirm the virtual environment is activated first: `source .venv/Scripts/activate`. Running `pip` or `python` outside the venv will use the wrong (or no) Python environment.

An annual report link shows `is_url_valid: false`

This can mean either the link is genuinely broken, or the source site (confirmed for `bseindia.com`) blocks automated requests outright regardless of headers. The API already sends a realistic browser User-Agent to reduce false negatives, but this distinction isn't fully resolvable without a real browser-based check — treat a false flag as "needs a manual look", not definitively broken.

7. Known, Documented Data Limitations

These are confirmed, investigated findings — not open bugs. Listed here so they're not mistaken for platform errors during normal use.

- **8 companies** (ULTRACEMCO, UNIONBANK, UNITDSPR, VBL, VEDL, WIPRO, ZOMATO, ZYDUSLIFE) appear in source P&L/balance sheet/cash flow files but have no entry in the master company list, so they're correctly excluded — the platform covers 92 of the real 100 Nifty 100 companies.
- **SBIN** has no balance sheet data; all balance-sheet-dependent ratios are correctly null for SBIN only.
- **10 sectors**, not the 11 referenced in some spec documents — confirmed repeatedly against the real data.
- **JIOFIN** has only 2 years of history, so 5-year metrics are null and it's skipped from batch tearsheet generation (91 of 92 tearsheets exist, not 92).
- **HAL, BEL, INDIGO** have mathematically correct but extreme ROE/ROCE figures due to a small equity base — excluded from ROE/ROCE-based screening, ranking, and averaging throughout the platform.
- **CIPLA and COALINDIA**'s operating_profit figures are unusually large relative to net profit — flagged for review, not yet resolved; noted here in case it affects a scoring-sensitive use case.
- **TCS's** stored roe_percentage field (0.52) is a raw decimal rather than a percentage; the platform's own computed ROE for TCS (in financial_ratios) is correct and unaffected — only the raw source field is off, and only for TCS.

For the full history of how each of these was found and confirmed, see the sprint retrospectives in notebooks/.